

WEEK 3 PRACTICE EXERCISES

Algorithms, Pseudocode, and Counting Basic Operations

Sections 1–3: Definition of Algorithms, Algorithm Representation, and Complexity of Algorithms

Name: _____ Student ID: _____

Counting convention. Unless stated otherwise, count each comparison, assignment, or execution of a statement as one basic operation. For FOR loops, do not count loop-control operations separately. The main goal is to determine how many times important statements are executed and then build the running-time function $T(n)$.

Exercise 1. Single loop with a condition

```

FUNCTION COUNT_POSITIVE
INPUT: array A of length n
OUTPUT: number of positive values

    count = 0

    FOR i = 0 TO n - 1 DO
        IF A[i] > 0 THEN
            count = count + 1

    RETURN count

```

1. How many times is the condition $A[i] > 0$ evaluated?
2. Suppose exactly p elements of A are positive. How many times is `count = count + 1` executed?
3. Count the following basic operations:
 - `count = 0;`
 - comparison `A[i] > 0;`
 - `count = count + 1;`
 - `RETURN count.`

Write the running-time function $T(n, p)$.

4. Find $T_{\text{best}}(n)$ and $T_{\text{worst}}(n)$. Clearly state what kind of input gives each case.

Exercise 2. Find the maximum: same input size, different inputs

```

FUNCTION FIND_MAX
INPUT: non-empty array A of length n
OUTPUT: largest value

    maximum = A[0]

    FOR i = 1 TO n - 1 DO
        IF A[i] > maximum THEN
            maximum = A[i]

    RETURN maximum

```

1. How many times is the comparison $A[i] > \text{maximum}$ evaluated?

- Let u be the number of times the assignment `maximum = A[i]` is executed. Write $T(n, u)$.
- Calculate the number of basic operations for

$$A = [10, 8, 7, 3, 1].$$

- Calculate the number of basic operations for

$$A = [1, 3, 5, 8, 10].$$

- Which type of input gives the worst case? Write $T_{\text{worst}}(n)$.

Exercise 3. Two nested loops

```

FUNCTION COUNT_PAIRS
INPUT: array A of length n

    count = 0

    FOR i = 0 TO n - 1 DO
        FOR j = 0 TO n - 1 DO
            count = count + 1

    RETURN count

```

- For a fixed value of i , how many times does the inner loop execute?
- How many different values does i take?
- Complete the following table.

n	Inner loop per i	Number of values of i	Total executions
2			
3			
5			
n			

- How many times is `count = count + 1` executed?
- Counting initialization and return as one operation each, find $T(n)$.

Exercise 4. Nested loops with changing bounds

```

FUNCTION COUNT_UNORDERED_PAIRS
INPUT: array A of length n

    count = 0

    FOR i = 0 TO n - 2 DO
        FOR j = i + 1 TO n - 1 DO
            count = count + 1

    RETURN count

```

- Complete the table below.

Value of i	Number of values of j
0	
1	
2	
$\vdots$	$\vdots$
$n - 2$	

2. Show that the total number of executions of `count = count + 1` is

$$(n - 1) + (n - 2) + \cdots + 2 + 1.$$

3. Use

$$1 + 2 + \cdots + (n - 1) = \frac{n(n - 1)}{2}$$

to obtain the total number of executions.

4. Counting initialization and return as one operation each, find $T(n)$.
 5. Verify your result manually for $n = 4$ by listing all pairs (i, j) processed by the algorithm.

Exercise 5. Repeated halving

```

FUNCTION HALVING
INPUT: positive integer n

    remaining = n
    count = 0

    WHILE remaining > 1 DO
        remaining = remaining DIV 2
        count = count + 1

    RETURN count
  
```

Assume first that

$$n = 2^k.$$

1. Trace the algorithm for $n = 16$.

Iteration	remaining before	remaining after
1		
2		
3		
4		

2. How many loop iterations are performed for:

$$n = 8, \quad 16, \quad 32, \quad 64?$$

3. Complete:

$$n = 2^k \quad \implies \quad k = \underline{\hspace{2cm}}.$$

4. Hence, how many times does the loop body execute as a function of n ?

5. Now count:

- `remaining = n`: 1 operation;
- `count = 0`: 1 operation;
- each test of `remaining > 1`: 1 operation;
- each statement inside the loop body: 1 operation;
- `RETURN count`: 1 operation.

Find the exact running-time function $T(n)$ for $n = 2^k$.

Focus on the counting process: Algorithm $\rightarrow$ number of executions $\rightarrow T(n)$.